

Project Report - ‘Autonomous ML Research Agent for Disaster Tweet Classification’

Track A — NLP with Disaster Tweets | Advanced Predictive Analytics T4 2025/2026

Francisca Nunes, Tiago Ferreira, Lourenço Sarmento, José Lopes

2026-05-20

Contents

0	Abstract	3
1	Introduction	3
2	Data Pre-Processing	3
3	Agent Architecture	4
3.1	The Loop	4
3.2	Module Structure	4
3.3	Key Design Decisions	5
4	Experiment Log and Iterative Tuning	5
4.1	Full Run Summary	5
4.2	Development Progression and Agent Improvements	7
4.3	Iterative Tuning	7
5	Pre-trained Models, Ensemble Classifier and Final Submission	8
5.1	DistilBERT	8
5.2	Roberta-base	8
5.3	First Ensemble	9
5.4	Final Ensemble and Kaggle Submission	10
6	Reflections on Course Content	10
7	Limitations and Future Work	11
8	Individual Contributions	12
9	Conclusion	12
10	References	13

11 Appendix	13
11.1 Tables	13
11.2 Materials for testing and analysis	15

0 Abstract

We built an autonomous ML research agent for the Kaggle Disaster Tweets task. The system uses a locally hosted LLM through Ollama to propose model ideas, generate executable Python code, run experiments, parse validation results, store memory, repair failures, and produce a Kaggle submission. The final agent follows a 10-step curriculum covering TF-IDF baselines, Keras CNN/RNN/Transformer models, and exploitation stages. We also compared the autonomous agent with manually fine-tuned pretrained models and an ensemble, which helped identify the main limitations and future directions for the system. The final agent completed autonomous runs, logged experiments in `experiments.json`, generated `analysis.txt`, and produced a Kaggle submission.csv. The [Appendix](#) includes links to the GitHub repository, experiment logs, notebooks, and submission files used to support the analysis and interpretation of the results.

1 Introduction

The task is Track A, [NLP with Disaster Tweets](#): a Kaggle binary classification problem with approximately 7,600 training tweets and 3,200 test tweets, evaluated using F1 score. Twitter (now X) has become an important communication channel during emergencies, but not every tweet that mentions “fire” or “crash” refers to a real disaster. The Disaster Tweets dataset therefore presents a non-trivial NLP challenge: tweets are short, noisy, and often contain disaster-ambiguous language. For example, “ablaze” may describe a real wildfire or be used metaphorically, while “flood” can refer to an information flood as easily as a natural disaster. The task requires models that capture context and intent, not just keyword presence.

The goal of this project was not to train the best model manually, but to build a machine that builds machines: an autonomous ML research agent. The agent uses a locally hosted LLM as its reasoning core to design, write, execute, evaluate, and iterate on machine learning experiments without human input. Given only the task description and a memory of previous results, the central question is whether the LLM can make useful experimental decisions over time, or whether it simply proposes disconnected model variations. Our system tests this by running a full loop: selecting an architecture family, generating executable Python code, executing it in a subprocess-based sandbox, parsing validation results, logging experiment memory, feeding the history back to the LLM, and finally producing a Kaggle submission file.

2 Data Pre-Processing

To keep the agent’s experiments comparable, the executor injects the same preprocessing helpers into every generated script: `clean_text()` and `make_features()`. This prevents each LLM-generated model from cleaning the tweets in a different way.

`clean_text()` applies basic tweet normalization. It lowercases text, removes URLs and HTML artifacts, strips emojis and other non-ASCII characters, expands common contractions, removes most punctuation, and collapses repeated whitespace. This reduces vocabulary noise and keeps the input representation consistent across experiments. Hashtags and mentions are preserved because they may contain useful task-specific signal, such as `#wildfire`. Contractions are expanded before punctuation is removed, so “don’t” becomes “do not” rather than “don t”.

`make_features(df)` builds the model input by concatenating the cleaned keyword field with the cleaned tweet text. The keyword often provides useful context about the tweet topic, so it is included directly in the text representation. Missing keywords are replaced with an empty string. The location field is not used because it is sparse and inconsistently formatted.

All experiments use the same 80/20 stratified train-validation split with `random_state = 42`. The prompts instruct the LLM to fit vectorizers and tokenizers only on the training partition to avoid validation leakage. The training set is mildly imbalanced, with approximately 57% non-disaster tweets and 43% disaster tweets. To reduce majority-class bias, the `system_prompt` instructs the LLM to use `class_weight = ‘balanced’` for sklearn models and `class_weight = {0: 1.0, 1: 1.5}` for Keras models.

Tokenizer	Unit	Vocab	OOV Handling	Used by
TF-IDF word	Words / n-grams	20k–50k features	Unseen terms ignored	LR, MLP, SVC
TF-IDF <code>char_wb</code>	Character 3–5-grams	20–50k features	Captures substrings	Classical TF-IDF refinements
Keras Tokenizer	Word indices	20k tokens	<OOV> token	CNN, GRU, Transformer
Pretrained tokenizers	Subwords	30,522	Splits rare words into subwords	Manual DistilBERT/RoBERTa baselines

3 Agent Architecture

3.1 The Loop

The agent follows a **propose** → **generate** → **execute** → **evaluate** → **log** → **iterate** cycle. It communicates with a locally hosted LLM via Ollama’s OpenAI-compatible API (localhost:11434/v1), making it fully offline with no API costs. The final agent code is model-agnostic and uses `AGENT_MODEL` to select the local Ollama model at runtime. If no model is specified, it defaults to `gemma4:latest`; however, the main evidence run reported in this paper used `qwen2.5-coder:14b` through Ollama. Other local models were tested during development, but they are not the main reported run. Temperature was set to 0.3, low enough for consistent and structured code generation but with sufficient variability to explore different architectural proposals. `Agent_max_tokens` was set to 8,192 to ensure complete training scripts are never truncated. Each iteration starts with the curriculum selector choosing an architecture family. The `proposed_prompt` then sends the LLM the assigned family, current best result, a stratified view of previous experiments, tried architectures, failed architectures, low-scoring architectures, compact hyperparameter signatures, and variation axes already explored. The LLM returns a complete Python training script, which the executor writes to a temporary file and runs in a subprocess-based sandbox with a four-minute timeout. If the script crashes, the agent asks the LLM to repair it for up to three attempts; on the final repair attempt, it requests a fresh rewrite instead of continuing to patch broken code. Once the script runs, the parser extracts the validation F1 and any Keras learning-curve lines, and the result is saved to `experiments.json` for the next iteration.

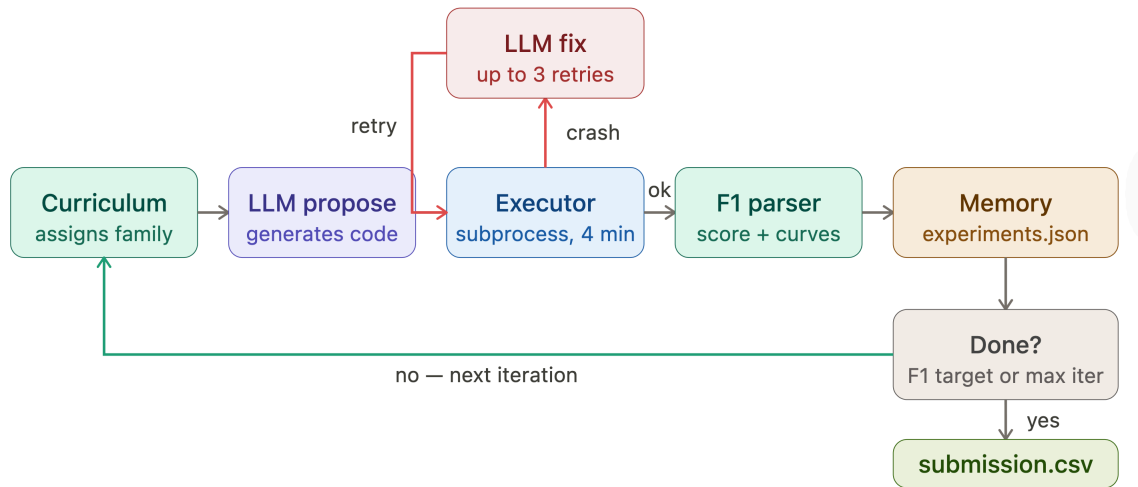


Figure 1: Agent Cycle

3.2 Module Structure

Module	Responsibility
<code>agent.py</code>	Main loop, curriculum scheduling, propose/fix/log orchestration, final LLM analysis, submission trigger
<code>llm.py</code>	Ollama OpenAI-compatible client; 3-retry backoff on transient errors; model-agnostic
<code>executor.py</code>	Subprocess sandbox; temp-file execution; <code>DATA_DIR</code> , cleaning helpers, imports, and Keras epoch-logger preamble injection
<code>prompts.py</code>	System prompt with Keras template + Gotchas; plus propose, fix and analyze templates
<code>parser.py</code>	Regex extraction: code blocks, architecture names, F1, <code>VARIATION_AXIS</code> , and <code>EPOCH_METRIC</code> learning curves
<code>memory.py</code>	<code>experiments.json</code> read/write; stratified history formatting; tried, failed, low-scoring, and best-run memory
<code>submission.py</code>	Kaggle <code>submission.csv</code> generation by rerunning best code, with <code>sklearn</code> /Keras fallback suffixes

3.3 Key Design Decisions

Prompt engineering. The `system_prompt` is one of the most important engineering artifacts in the project. It defines the task, allowed and forbidden libraries, known Keras/sklearn gotchas such as `MultiHeadAttention` requiring the Functional API in Keras 3 and `LinearSVC` not accepting `n_jobs`, strict output-format rules, and data hygiene rules such as using an 80/20 split before fitting and fitting preprocessing only on the training partition. The `propose_prompt` receives the curriculum-assigned family, current best result, tried architectures, failed architectures, low-scoring architectures, compact hyperparameter signatures, and the variation axes already explored. This matters because the LLM is not proposing from scratch each time: it can see what has already worked, what crashed, and what should not be repeated without a meaningful change.

Sandboxed execution. Code is written to a temp file and launched as a subprocess, never through `eval()` or `exec()`. The executor preamble injects `data_dir`, `clean_text()`, `make_features()`, common scientific and sklearn imports, and a Keras monkey-patch that automatically attaches an `EpochPrinter` callback to every `model.fit()` call. The callback prints one parseable `epoch_metric` line per epoch containing training loss and validation loss, which the parser extracts to build learning curves. This design means the LLM does not need to instrument its own training code, reducing a common source of generated-code errors.

Curriculum and memory. The 10 curriculum steps map the experiment sequence from classical baselines to neural architectures and then exploitation/refinement: TF-IDF linear $\rightarrow$ probabilistic $\rightarrow$ Dense MLP $\rightarrow$ CNN $\rightarrow$ GRU/BiLSTM $\rightarrow$ CNN+GRU $\rightarrow$ Transformer $\rightarrow$ exploit classical $\rightarrow$ exploit deep $\rightarrow$ final refinement. Memory in `experiments.json` stores the prompt, generated code, architecture, assigned family, status, F1, stdout/stderr snippets, timestamp, learning curves, variation axis, and the LLM’s own analysis of each iteration. The history formatter builds a stratified memory view: best runs overall, all runs in the current family, tried architectures, failed architectures, low-scoring architectures, compact hyperparameter signatures, and variation axes already explored. During exploitation/refinement, the agent can also reuse the previous best model code as a baseline and ask the LLM to change one meaningful design choice. This makes later proposals genuinely informed by prior experiments rather than disconnected attempts.

4 Experiment Log and Iterative Tuning

4.1 Full Run Summary

Curriculum Family	Best Architecture in Log	Best Val. F1
TF-IDF strong linear baseline	TF-IDF + <code>LinearSVC</code>	0.7713
TF-IDF probabilistic baseline	TF-IDF + <code>ComplementNB</code>	0.7637

Curriculum Family	Best Architecture in Log	Best Val. F1
TF-IDF Dense MLP	TF-IDF + <code>MLPClassifier</code> with activation/hidden-unit changes	0.7591
Keras Embedding + 1D CNN	Keras Embedding + 1D CNN	0.7721
Keras Embedding + GRU or BiLSTM	Keras Embedding + GRU	0.7732
Keras Embedding + CNN + GRU/BiGRU hybrid	Keras Embedding + CNN + GRU hybrid	0.7920
Keras Transformer / self-attention	Keras small Transformer / self-attention block, increased maxlen	0.7723
Exploit best classical family	TF-IDF + <code>LinearSVC</code> , increased <code>max_features</code>	0.7733
Exploit best deep-learning family	Keras CNN + GRU hybrid, modified dropout	0.7719
Final best-model refinement	Keras CNN + GRU hybrid, modified dropout	0.7717

Because the final log contains 30 experiments, this table summarizes the best result observed within each curriculum family rather than listing every individual experiment.

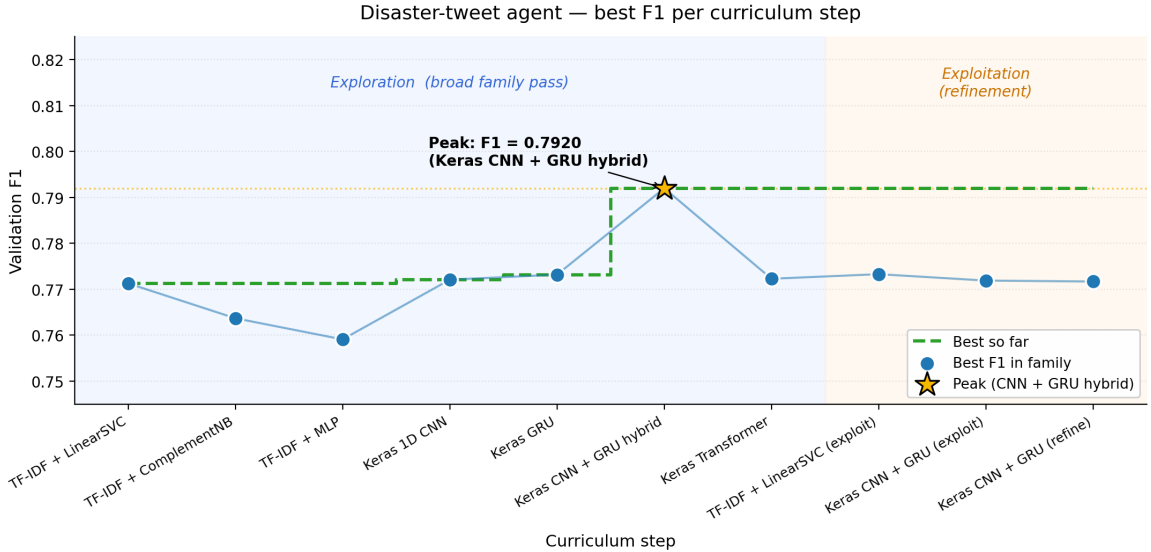


Figure 2: Best validation F1 reached by the agent at each curriculum step in the final run, per family

A concrete example of memory directing a winning proposal occurs at iteration 16. The agent’s log showed the GRU (iteration 13, $F1 = 0.7732$) had outperformed the standalone CNN (iteration 10, $F1 = 0.7721$), but its own analysis had flagged that neither architecture alone was capturing the full signal. With both results in memory, the agent proposed combining them into a hybrid: a 128-dimension embedding feeding a Conv1D into MaxPooling and then a GRU, with 0.5 dropout and class weighting. The result was $F1 = 0.7920$, the best autonomous score of the entire run, and a gain the agent attributed directly to the two layer types working in sequence rather than either working alone.

The curriculum was built to make the agent explore before it exploits. The first seven steps push it through the main families we wanted to test: TF-IDF linear models, probabilistic baselines, dense MLPs, CNNs, recurrent models, CNN-RNN hybrids, and small self-attention models. Only after this broad pass does the agent move into exploitation: one step focuses on the best classical approach, one on the best deep-learning approach, and the final step refines whichever model family looks strongest overall.

This was a practical design choice. With our hardware and project timeline, a full open-ended search was not realistic: one complete run needed to stay around one hour, so the agent could not run large grid searches or repeatedly

tune every architecture. The curriculum gives the search some discipline. It makes sure the agent sees the main model families first, then spends the remaining budget on the most promising directions instead of wandering randomly.

4.2 Development Progression and Agent Improvements

Initial autonomous loop → Compatibility fixes → Leakage prevention → Curriculum and memory → Executor robustness → Exploitation logic → Controlled tuning

The final agent was built through several rounds of testing and refinement. The first working version already had the basic autonomous loop: prompt the local LLM, extract generated code, execute it, parse the F1 score, log the result, and generate a submission. Early runs showed that this simple loop was not enough for reliable autonomous experimentation. Generated scripts could miss imports, use incompatible sklearn or Keras arguments, fit preprocessing on the wrong data split, or return outputs in formats the parser could not read consistently.

These issues guided the next improvements. We strengthened the `system_prompt` with stricter rules about available libraries, validation leakage, output format, and known sklearn/Keras gotchas. The executor was also improved with a shared preamble that injects `data_dir`, `clean_text()`, `make_features()`, common imports, and the Keras epoch logger into every generated script. This made the generated code more consistent and reduced repeated failure modes.

The final stage was to make the search process more structured. We added the curriculum so the agent would not randomly jump between models but would first explore the main families covered in the course and then move into exploitation. We also expanded memory so each new proposal could use a stratified view of previous results: the best runs overall, all runs in the currently assigned family, tried architectures, failed architectures, low-scoring architectures, compact hyperparameter signatures, and the variation axes already explored. This made the agent less likely to repeat near-duplicate experiments and helped it make more meaningful changes during later iterations. In this way, the agent improved from a simple code-generation loop into a more reliable autonomous research system that could explore, learn from its logs, repair errors, and produce a Kaggle submission with limited human intervention.

4.3 Iterative Tuning

The last tuning step was less about making the agent search more, and more about making it search better. In earlier versions, the LLM sometimes changed too many things at once, which made it hard to tell whether a result improved because of the model family, the vectorizer, the regularization, or a training setting. In the final version, the `PROPOSE_PROMPT` asks the LLM to declare a `VARIATION_AXIS`, avoid near-duplicate configurations within the assigned family, and make controlled changes rather than broad rewrites. During exploitation/refinement, the controller selects the best prior model from the relevant family, injects its full code into the prompt, and asks the LLM to change one meaningful design choice.

For classical models, most tuning focused on the TF-IDF representation and Logistic Regression regularization. The final prompt constrained TF-IDF models to use `max_features` between 20,000 and 50,000, `ngram_range` = (1,2) or (1,3), and `sublinear_tf` = True. For Logistic Regression, the LLM could explore C values from {0.1, 0.5, 1.0, 5.0, 10.0} and different class-weight settings. This gave the agent room to improve strong classical baselines without turning the run into a slow grid search.

For Keras models, we narrowed the search to a few settings that mattered most for runtime and generalization. The final prompt made these choices explicit: embedding dimension could be 64 or 128, dropout could be 0.2, 0.3, or 0.5, EarlyStopping patience could be 3 or 4, and batch size could be 64, 128, or 256. Sequence length was fixed at `maxlen=40`, and Keras runs used `class_weight` = {0: 1.0, 1: 1.5}. This kept neural experiments feasible while still letting the agent compare CNNs, GRU/BiLSTM models, CNN-GRU hybrids, and self-attention models.

The experiment log shows why this controlled tuning was useful. Across 30 experiments, the agent completed 30 successful runs and captured 17 Keras learning curves, so we could see more than just the final F1 score. The strongest autonomous result in the final repository log came from the CNN-GRU hybrid family, while Keras 1D CNN, GRU, Transformer/self-attention, and TF-IDF LinearSVC variants remained competitive. More complex models were still worth testing, but they did not automatically perform better. The main lesson was practical: after each run, the agent should not just add complexity; it should make small, interpretable changes inside a structured curriculum.

5 Pre-trained Models, Ensemble Classifier and Final Submission

Our approach to the challenge followed the two main goals defined as guidelines for the project, where we: 1) built an AI-powered autonomous research agent that designed, trained, evaluated, and iterated over deep learning models - without human intervention; 2) tuned and tested several locally hosted Large Language Models as its “brain” to drive the machine learning experimentation.

However, as we dug deeper into the competition, we became aware of several pre-trained transformer encoder models that could be used to better identify target tweets. By researching these models, we found out they could not be implemented within our locally run LLM’s. Nevertheless, we tested them to serve as benchmarks for improvement in accuracy metrics such as the F1 score and to complement the models selected by our local LLM agent, whose main reported run used qwen2.5-coder:14b through Ollama.

Based on our restrictions of *a) limited computing power* and *b) a limit of 1 hour for the program to run*, we tested two models: 1) DistilBERT and 2) RoBERTa. They proved to run efficiently and achieved good results. These models were initially run in parallel to our locally hosted LLM agent.

5.1 DistilBERT

DistilBERT (Sanh et al., 2019) is a knowledge-distilled version of BERT-base. It is substantially smaller and faster than the original, while retaining most of BERT’s downstream performance (see [Table 5.1 in the Appendix](#)). Crucially, it enters fine-tuning with a rich prior over English semantics — it already understands that “raging” frequently precedes disaster language, and that metaphorical disaster usage (“flood of emails”) sits in a different syntactic context than literal usage. Our from-scratch models must infer all of this from ~6,000 examples.

Unlike our other models, raw uncleaned tweet text is fed directly to DistilBERT’s built-in preprocessor. The WordPiece tokenizer was designed for raw, capitalized, punctuated text, and capitalization carries a meaningful signal (“BREAKING” in disaster tweets) that our `clean_text()` pipeline would remove.

We fine-tuned the model via KerasNLP using the same 80/20 stratified split as all agent experiments; the full fine-tuning configuration, hardware and results are reported in [Table 5.1 in the Appendix](#). Train and validation metrics tracked closely across both epochs, showcasing no significant overfitting.

This result served as a benchmark for our local agent, defining the target it should try to achieve through model selection and iteration, as DistilBERT represents the most rudimentary pre-trained model applicable in this competition setting. However, no matter how many iterations or prompt/variable adjustments we made, the locally run LLM never matched the result of this first model trial.

The clear performance advantage confirmed that pretrained encoders represent a qualitatively different tier: no amount of classical or Keras tuning closed this gap.

5.2 Roberta-base

RoBERTa (Liu et al., 2019) is a robustly optimized re-pretraining of BERT-base that keeps the original architecture but reworks the pretraining recipe: it removes Next Sentence Prediction, applies dynamic masking, uses roughly ten times more text, and trains for substantially longer with larger batches. It also swaps WordPiece for byte-level Byte-Pair Encoding, which handles the noisier surface features of social media (hashtags, mentions, mixed-case shouting, punctuation runs) more gracefully than DistilBERT’s WordPiece splits. The result is a stronger semantic prior on casual, capitalized English at the cost of roughly twice DistilBERT’s parameter count and a slower forward pass. The full architecture and pre-training details are summarized in [Table 5.2 - Appendix](#).

As with DistilBERT, raw tweet text was fed directly to RoBERTa’s tokenizer rather than the output of our `clean_text()` pipeline. The byte-level BPE encoder is designed for raw, cased text and treats punctuation and capitalization as meaningful subword signals; pre-cleaning would strip exactly the cues the pretrained weights have learned to exploit.

We fine-tuned the model via KerasHub, using EarlyStopping on validation loss and a mild class weighting to balance recall on the under-represented disaster class, on the same 80/20 stratified split used for every other experiment. To make a fine-tune at this parameter count tractable on the M4 MacBook Air, training ran on the Apple Silicon GPU

through Keras 3’s PyTorch (MPS) backend rather than on the CPU path used for DistilBERT. All hyperparameters and the final result are listed in Table 5.2. We also swept the decision threshold over the softmax probabilities of the disaster class; the default 0.50 cut-off already sat at the F1 maximum, which is the expected behaviour for a well-calibrated fine-tuned encoder on a near-balanced label distribution, so no post-hoc threshold tuning was applied to the test submission.

We initially attempted DeBERTa-v3-base, currently the strongest encoder in this size class on GLUE/SuperGLUE, but its disentangled-attention kernels crashed repeatedly on Apple MPS, so RoBERTa-base, which uses standard scaled-dot-product attention, became our top transformer benchmark under the local-hardware constraint.

As with DistilBERT, no amount of prompt engineering, few-shot re-design or model stacking by our locally hosted agent closed the gap to this number; the pretrained-encoder tier remained out of reach for a zero or few-shot generative model running through Ollama.

5.3 First Ensemble

As previously mentioned, the pre-trained encoder models drastically outperformed our locally hosted LLM agents, no matter the prompt engineering applied. Facing this scenario, in order to achieve more accurate predictions for the Disaster Tweets Classification, we came up with the idea to test an Ensemble model. From our previous experience in classification tasks, we know that an Ensemble of several models usually yields higher results than each model separately.

In order to respect the hardware and time limits imposed, to create the Ensemble we opted to go with the following architecture: three models, trained on an identical 80/20 stratified split, each producing a calibrated probability of the disaster class on both the validation and test sets. Those probabilities are then linearly combined and a single decision threshold is applied through a soft-voting ensemble. The choice of constituents was deliberate: one classical baseline, one mid-tier deep model, and the strongest pretrained encoder we could run locally, with both the classical baseline and the deep model coming from the respective highest F1 scores our local LLM agent provided. This diversity is what gives Ensemble its lift, near-identical models add almost nothing.

Model 1 - *CountVectorizer + Logistic Regression.*

This is the architecture our locally hosted agent kept rediscovering as its leaderboard winner across runs (internally labelled count_logreg_trigrams). A scikit-learn Pipeline of a CountVectorizer with word n-grams from 1 to 3, min_df = 2 and max_features = 30,000, feeding a LogisticRegression with C = 1.0, max_iter = 2,000, class_weight = “balanced” and the liblinear solver. The model trains in seconds on CPU and reached validation F1 = 0.7755 on this run — the highest score the agent ever produced without using any pre-trained weights.

Model 2 - *Keras Embedding + custom Transformer encoder.*

The agent’s runner-up architecture: a single transformer encoder block built from Keras layers, with a learned 128-dimension embedding over a 20,000-token vocabulary. The block follows a classical 4-head multi-head self-attention, residual + LayerNorm, position-wise feed-forward of width 256 with ReLU, residual + LayerNorm. The encoded sequence is mean-pooled, passed through a 64-unit ReLU head with 0.2 dropout, and projected to a single sigmoid output. Validation F1 = 0.7407.

Model 3 - *RoBERTa-base fine-tuned.*

Identical configuration to Section 5.2, preset roberta_base_en, sequence length 160, three epochs guarded by EarlyStopping on validation loss, run on the Apple Silicon GPU through Keras 3’s PyTorch (MPS) backend. On our final submission run it returned validation F1 = 0.8183, within run-to-run noise of the 0.819 reported in 5.2, which confirms the headline RoBERTa result is stable.

Once the three validation probability vectors were in hand, we ran a joint grid search over the weight simplex and the decision threshold: - RoBERTa weight (a) was swept from 0.50 to 0.95 in 10 steps - Transformer weight (b) from 0 to (1 - a) in 8 steps; - LogReg weight c = 1 - a - b was fixed by the constraint so the weights always sum to one; - Threshold (t) was swept from 0.30 to 0.70 in 21 steps (0.02 resolution). - for each of the ~1,680 candidate configurations we computed the ensemble probability on the validation set and recorded the F1 of (p t). - the maximizing combination was a = 0.900, b = 0.057, c = 0.043, t = 0.56; placing almost all of the predictive weight on

RoBERTa but allowing the two weaker models to break ties on borderline tweets.

The weighted ensemble reached validation $F1 = 0.8321$, a gain of $+0.0138$ over the best single model (RoBERTa at 0.8183) and our highest score on the disaster-tweets validation set so far. The same weights and threshold were then applied to the test-set probabilities to generate `submission_final_ensemble.csv` (2,012 predicted non-disaster, 1,251 predicted disaster). It achieved a Kaggle competition score of 0.83849, our best result at the moment. A summary of the ensemble setup, the constituent models and the final result is given in Table 5.3 - Appendix.

5.4 Final Ensemble and Kaggle Submission

Building on the previous ensemble, we revisited the pipeline to make the two cheaper models more stable without paying the cost of running RoBERTa multiple times. A full 5-fold cross-validation across all three models would have been the textbook answer, but each RoBERTa fold takes several minutes on the MPS backend and pushed the notebook past our self-imposed runtime budget. We therefore adopted a hybrid recipe: fold-average the two cheap models, run RoBERTa only once on the same 80/20 split, and re-tune the soft-voting weights on the shared validation set.

For each of the five stratified folds, a fresh CountVectorizer + LogReg and a fresh Keras Transformer (identical architectures to the first ensemble) were trained on the fold’s training partition and used to predict on the shared validation and test sets. The five probability vectors were averaged per model. We then trained one extra pass of each cheap model on the full 6,090-row training split and blended the CV-averaged and full-data probabilities. Best blends were 80/20 (LogReg, $F1 = 0.7786$) and 60/40 (Transformer, $F1 = 0.7830$), both slightly above the single-split scores of the first ensemble. RoBERTa was fine-tuned once on the 80/20 split with the exact configuration of Section 5.2 (validation $F1 = 0.8238$) and a second time on all 7,613 labelled rows with no validation set; the test-set probabilities of the two runs were averaged 50/50. The full-data refit never enters the validation grid search as, by construction, it has seen everything and exists solely to strengthen the final test predictions.

The weight/threshold grid search ran in steps of 0.025 over the full simplex and 121 steps over the threshold (0.005 resolution). The maximizing combination was $a = 0.675$ (RoBERTa), $b = 0.125$ (Transformer), $c = 0.200$ (LogReg), $t = 0.645$ — redistributing weight away from RoBERTa ($0.900 \rightarrow 0.675$) and giving more credit to the two now-smoothed cheap models. The weighted ensemble reached validation $F1 = 0.8322$, almost numerically tied with the first ensemble but produced by a meaningfully different vector. A summary of the setup and final outcome is given in Table 5.4 - Appendix.

Applied to the test set, it generated `submission_cheaper_cv_ensemble.csv` (2,059 predicted non-disaster, 1,204 predicted disaster) and scored 0.84370 on the [competition leaderboard](#), improving on the 0.83849 of the first ensemble and becoming our best result of the project. The final competition score has placed us at ‘#65’ out of ‘#713’ teams on the leaderboard, where the top ~40 entries score 1.000 due to a long-standing public-test-set label leak on this competition; we treat the next tier as the realistic ceiling, thus placing our team in the top ‘#25’. This gap between local validation $F1$ (0.8321) and Kaggle score (0.84370) is largely explained by the full-data RoBERTa refit: it was trained on all 7,613 labelled rows and contributes to the test predictions, but by construction never enters the validation grid search. Therefore, its lift can only materialise on the leaderboard.

6 Reflections on Course Content

Table — Course topics mapped to project use and findings

Course Topic	Project Use	Finding
Bag-of-words / n-grams + MLP	TF-IDF baselines	Strong and fast
Dense MLPs	MLP on TF-IDF	Useful but weaker
1D CNNs	Keras Embedding + Conv1D	Strong neural baseline
RNNs / GRU / BiLSTM	Keras sequence models	Competitive, less stable
Learned embeddings	Keras Embedding layer	Worked with simple CNNs
Self-attention	Keras <code>MultiHeadAttention</code>	Explored, not dominant

Course Topic	Project Use	Finding
Pretrained encoders	DistilBERT / RoBERTa	Strong manual ceiling

The project connected most directly with the course content on text representation and neural sequence modelling. The agent’s curriculum followed the progression from sparse text representations to learned neural representations: TF-IDF n-grams, dense MLPs, CNNs, recurrent models, and self-attention. This made the search space course-driven rather than arbitrary.

The main lesson was that complexity alone did not guarantee better performance. TF-IDF models remained strong because bag-of-words and n-gram features are very effective for short text classification. At the same time, the strongest autonomous neural result came from compact Keras sequence models, especially the CNN-GRU hybrid, while the 1D CNN remained a strong simpler baseline.

The RNN and GRU/BiLSTM experiments are also connected naturally to the course content because they model sequence order. However, they are harder to train and do not consistently beat simpler alternatives on short tweets, mainly due to short tweets having no long-range dependencies to exploit. Similarly, the Keras self-attention model implemented the Transformer idea, but without pretraining it did not show the same advantage as pretrained encoders.

The manual DistilBERT and RoBERTa baselines helped clarify this distinction. Their advantage came not only from the Transformer architecture, but from pretrained language knowledge. This was one of the clearest reflections from the project: for small datasets, pretrained representations can matter more than adding a more complex architecture from scratch.

7 Limitations and Future Work

This section is organised around the concrete things the autonomous agent could not accomplish under our setup, and the design change each would require in a follow-up version. The 0.8321 validation F1 reported in Section 5.3 is a strong result on this task with the hardware available to us, but it is the manual ceiling we reached, not the ceiling the agent reached on its own. Across the report, the dominant signal is the gap between the agent’s autonomous best ($F1 = 0.7920$, Section 4) and what we produced manually with pretrained encoders and Ensemble ($F1 = 0.819$ for RoBERTa in 5.2, $F1 = 0.8321$ ensembled in 5.3). Each theme below is, in some form, a restatement of that gap, a thing the agent could not reach inside its time, hardware or memory budget, together with the design change that would let a successor agent reach it.

The agent could not fine-tune pretrained transformer encoders within its run budget. The DistilBERT, RoBERTa-base, and ensemble RoBERTa runs described in Section 5 exceeded the four-minute experiment timeout by roughly an order of magnitude, while the one-hour total run budget barely accommodated a single fine-tuning attempt. As a result, the strongest results could not be produced autonomously by the agent. Future work should introduce a separate “slow lane” with 30–60-minute timeouts, GPU access, and a longer total budget, enabling the agent to complete encoder fine-tunes without human intervention.

The agent also ran all deep-learning experiments on CPU, limiting the performance of from-scratch neural models. Because the executor lacked GPU support, the Keras CNN, GRU/BiLSTM, and small-Transformer models in Section 4 plateaued around 0.77–0.79 F1 and did not close the gap to the pretrained encoders and ensemble. Future work should expose GPU devices to the executor and configure the Keras backend accordingly. This infrastructure change would allow longer and more effective training, especially for deep-learning curriculum steps.

A further limitation comes from the agent’s own tuning strategy rather than its environment. As described in Section 4.3, the `propose_prompt` restricts each new experiment to change at least one but no more than two hyperparameters within the assigned family. This was a deliberate design choice — it kept individual results interpretable and prevented the LLM from rewriting most of the script between iterations, so we could tell why a given experiment did or did not improve — but it also bounded how aggressively the exploitation rounds could search. The flat best-so-far line after step 4 in Figure 2 is partly a consequence of this: with only one or two hyperparameters moving per step, the agent could not jointly tune embedding dimension, dropout, batch size, and learning rate in a single experiment,

which is what a coarser grid or Bayesian search would do. Future work should relax this rule for the exploitation phase only — preserving the two-parameter limit during exploration for interpretability, but allowing a wider, joint hyperparameter sweep on the two strongest families once they have been identified.

Local 7–14B LLMs appeared to reach a performance ceiling of approximately 0.79 validation F1 in our autonomous setup. Across the agent runs in Sections 3 and 4, prompt revisions and local model changes improved reliability, but did not push the autonomous agent beyond F1 = 0.7920 or close the gap to the encoder-based models reported in Section 5. These models generated valid code but showed weaker experimental judgement than a human practitioner. Future work should retain the existing agent scaffold while replacing the LLM with a larger reasoning-tuned model, such as Qwen2.5-72B-Instruct, Llama-3.3-70B-Instruct, or DeepSeek-R1.

The agent did not propose ensembling its own outputs. The Section 5.3 ensemble, which achieved the project’s best F1 score of 0.8321, was created manually by combining the best agent-built models with a hand-fine-tuned RoBERTa model. Future work should add an explicit ensembling stage to the curriculum, allowing the agent to select top-performing models from experiments.json, retrain them on the same split, and perform a small weight-and-threshold grid search. This would convert the curriculum’s model diversity into a measurable performance gain.

8 Individual Contributions

Table — Team member contributions

Team Member	Contributions
Francisca	Project skeleton; core agent; main loop; early runs; debugging; submission setup
Tiago	Prompt engineering; <code>SYSTEM_PROMPT</code> ; <code>PROPOSE_PROMPT</code> ; <code>FIX_PROMPT</code> ; preprocessing rules; tuning rules
José	DistilBERT; RoBERTa; pretrained baselines; course reflection; limitations; references
Lourenço	Curriculum; memory logic; experiment analysis; README checks; architecture section; final polishing

All members collaborated on testing, debugging, README preparation, and the final video. LLM-assisted tools were used for coding support, debugging, and writing assistance, while final design decisions, experiments, and report validation were carried out by the team.

9 Conclusion

This project delivered a working autonomous ML research agent for the Disaster Tweets task. What made it successful was the experimentation loop around the models: a local LLM proposes experiments, writes code, runs models, reads the results, stores memory, fixes failures, and produces a Kaggle submission. That was the real objective of the project: to move from manually testing models to building a system that can run part of the research process on its own.

The experiment log showed that autonomy only becomes useful when it has structure. The curriculum kept the agent grounded in the main NLP model families from the course, while memory and prompt constraints helped it avoid repeated mistakes and meaningless tuning. One of the clearest lessons was that the most complex model is not always the best one. On this dataset, simpler and better-controlled models often gave the best balance between F1, runtime, and reliability.

The manual pretrained and ensemble experiments helped us see what the next version of the agent should aim for. DistilBERT, RoBERTa, and the ensemble benefited from pretrained language knowledge and modelling choices that the current autonomous loop could not yet handle within its time and hardware limits. A future version should therefore include a slower advanced-model track, GPU-aware execution, and an automated ensembling step. Overall, the project showed that autonomous ML is not only about choosing architectures; it is about designing the whole feedback loop: prompts, memory, execution safety, logging, repair, and iteration.

10 References

- Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.
- Kaplan, J. et al. (2020). Scaling laws for neural language models. arXiv:2001.08361
- Hoffmann, J. et al. (2022). Training compute-optimal large language models. arXiv:2203.15556.
- Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT. arXiv:1910.01108.
- He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training. arXiv:2111.09543.
- Meta AI. (2025). Llama 4. <https://ai.meta.com/llama/>
- Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning.
- Kaggle. (2019). NLP with Disaster Tweets. <https://www.kaggle.com/competitions/nlp-getting-started>

11 Appendix

11.1 Tables

Table 5.1 — DistilBERT: model specification, fine-tuning setup and results

Category	Item	Value
Model	Architecture	Knowledge-distilled BERT-base
Model	Transformer layers	6 (vs 12 in BERT-base)
Model	Parameters	66M (−40% vs BERT-base)
Fine-tuning	Preset	<code>distil_bert_base_en_uncased</code>
Fine-tuning	Framework	KerasNLP (<code>from_preset()</code>)
Fine-tuning	Input	Raw tweet text (no <code>clean_text()</code>)
Fine-tuning	Epochs	2
Fine-tuning	Learning rate	1e−5
Fine-tuning	Batch size	32
Fine-tuning	Train / validation split	80 / 20, stratified
Hardware	Device	CPU — 2024 M4 MacBook Air, 16 GB RAM
Hardware	Training time	~45 minutes total
Results	Train accuracy	84.60%
Results	Validation accuracy	84.83%
Results	Validation loss	0.380
Results	Validation F1	0.810

Table 5.2 — RoBERTa: model specification, fine-tuning setup and results

Category	Item	Value
Model	Architecture	Re-pretrained BERT-base
Model	Transformer layers	12
Model	Parameters	~125M
Fine-tuning	Preset	<code>roberta_base_en</code>
Fine-tuning	Framework	KerasHub (<code>from_preset()</code>)
Fine-tuning	Input	Raw tweet text (no <code>clean_text()</code>)

Category	Item	Value
Fine-tuning	Epochs	3 (EarlyStopping on <code>val_loss</code> , <code>patience</code> = 1, <code>restore_best_weights</code> = True)
Fine-tuning	Optimizer	AdamW
Fine-tuning	Learning rate	1e−5
Fine-tuning	Weight decay	0.01
Fine-tuning	Loss	Sparse categorical cross-entropy (from logits)
Fine-tuning	Class weights	{0: 1.0, 1: 1.4}
Fine-tuning	Batch size	32
Fine-tuning	Train / val / test split	80 / 20 stratified — 6,090 / 1,523 / 3,263
Fine-tuning	Decision threshold	0.50 (sweep 0.20–0.80, step 0.02, confirmed optimum)
Hardware	Device	M4 MacBook Air — Apple Silicon GPU via Keras 3 PyTorch (MPS) backend
Results	Validation F1	0.819

Table 5.3 — Ensemble: setup, component models, grid search and best result

Category	Item	Value
Setup	Validation split	80 / 20 stratified, <code>random_state</code> = 42
Setup	Sample sizes	6,090 train / 1,523 val / 3,263 test
Setup	Combination rule	Linear weighted average of class-1 probabilities, then threshold
Model 1 — CountVec + LogReg	Validation F1	0.7755
Model 2 — Keras Embedding + Transformer	Validation F1	0.7407
Model 3 — RoBERTa-base	Configuration	Identical to Section 5.2 (preset <code>roberta_base_en</code>)
Model 3 — RoBERTa-base	Validation F1 (this run)	0.8183
Grid search	RoBERTa weight <i>a</i>	<code>linspace(0.50, 0.95, 10)</code>
Grid search	Transformer weight <i>b</i>	<code>linspace(0.0, 1 - a, 8)</code>
Grid search	LogReg weight <i>c</i>	$1 - a - b$ (constraint)
Best ensemble	Validation F1	0.8321

Table 5.4 — Final ensemble (cheaper CV variant): setup, components and outcome

Category	Item	Value
Setup	Validation split	80 / 20 stratified, <code>random_state</code> = 42 (shared with 5.1–5.3)
Setup	Sample sizes	6,090 train / 1,523 val / 3,263 test
Setup	Cross-validation	5 stratified folds on training partition, cheap models only
LogReg	CV / full blend	0.80 CV + 0.20 full
LogReg	Validation F1	0.7786 @ $t = 0.485$
Transformer	CV / full blend	0.60 CV + 0.40 full
Transformer	Validation F1	0.7830 @ $t = 0.480$
RoBERTa	Runs	$1 \times \text{val-split} + 1 \times \text{full-data}$ (test probs averaged 50/50)
RoBERTa	Validation F1 (val-split run)	0.8238 @ $t = 0.750$
Grid search	Weight step	0.025 over simplex $w_R + w_T + w_L = 1$

Category	Item	Value
Grid search	Threshold sweep	linspace(0.20, 0.80, 121)
Best ensemble	Weights (R / T / L)	0.675 / 0.125 / 0.200
Best ensemble	Decision threshold	0.645
Best ensemble	Validation F1	0.8321
Best ensemble	Gain over best single model	+0.0083 over RoBERTa (0.8238)
Submission	File	submission_cheaper_cv_ensemble.csv
Submission	Predicted class distribution	2,059 non-disaster / 1,204 disaster
Submission	Kaggle public score	0.84370 (vs. 0.83849 of submission_final_ensemble.csv)

11.2 Materials for testing and analysis

In this section we provide links for our final materials of the report and submission.

- GitHub Repository - <https://github.com/franciscanunes2001/APA>
- Disaster Agent folder - https://github.com/franciscanunes2001/APA/tree/main/disaster_agent
- Jupyter Notebooks folder - <https://github.com/franciscanunes2001/APA/tree/main/jupyter-notebooks>
- Video Demo of Agent Running - <https://github.com/franciscanunes2001/APA/blob/main/Video.mov>
- Best Model and Final Submission to Kaggle Competition - <https://github.com/franciscanunes2001/APA/blob/main/jupyter-notebooks/disaster-tweets-cheaper-cv-ensemble.ipynb>
- Kaggle Competition - <https://www.kaggle.com/competitions/nlp-getting-started>
- Kaggle Leaderboard, Team Name “APA T4 25/26 CLSBE - G_pt”- <https://www.kaggle.com/competitions/nlp-getting-started/leaderboard>